

Same Bits Without Losing MIPS

Reproducible Numerics at Full Hardware Speed

ANDREW DRAKEFORD

The Problem

Models generate different answers when:

- 1) Run on different hardware
- 2) Different number of threads
- 3) Different instruction sets
- 4) Built using different tool chains
- 5) And even from run to run

Moon Shot

Large Language Model llama2.c Running on:

- 1) Apple M4
- 2) x86 - Ryzen 9 7900X
- 3) CUDA - NVIDIA RTX 5070

Moon Shot

Large Language Model llama2.c Running on:

- 1) Apple M4
- 2) x86 - Ryzen 9 7900X
- 3) CUDA - NVIDIA RTX 5070

4) Bitwise reproducible

Moon Shot

Large Language Model llama2.c Running on:

- 1) Apple M4
- 2) x86 - Ryzen 9 7900X
- 3) CUDA - NVIDIA RTX 5070

4) Bitwise reproducible

5) Don't slow down

Results: bitwise equality

			threads			
	machine	toolchain	1	2	4	8
AVX2	Ryzen 9 7900X, Windows 11	MSVC 19.44	match	match	match	match
AVX-512	Ryzen 9 7900X, Windows 11	MSVC 19.44	match	match	match	match
NEON	Apple M4, macOS 15.5	Apple clang 17	match	match	match	match
CUDA	RTX 5070, Windows 11	nvcc 13.3				match

logits c50b9583a5fa7dc1
key 3fd8a79124deaa91
value 66ffa1de82398611

12,864 checkpoints hashed per run
13 configurations, 0 disagreements
Same token stream on every one

Results: performance, tokens per second

each cell is tokens per second at that thread count

Ryzen 9 7900X, Windows 11, MSVC 19.44

tokens/s	1 thread	2 threads	4 threads	8 threads
llama2.c, AVX2	131	239	502	605
deterministic, AVX2	479	571	695	673
llama2.c, AVX-512	133	262	503	610
deterministic, AVX-512	623	652	713	673

Apple M4, macOS 15.5, Apple clang 17

tokens/s	1 thread	2 threads	4 threads	8 threads
llama2.c	209	336	536	455
deterministic, NEON	304	493	673	633

Results: performance, tokens per second

Ryzen 9 7900X, Windows 11, MSVC 19.44

RTX 5070, CUDA 13.3

Deterministic, CUDA: 11.2 tokens per second

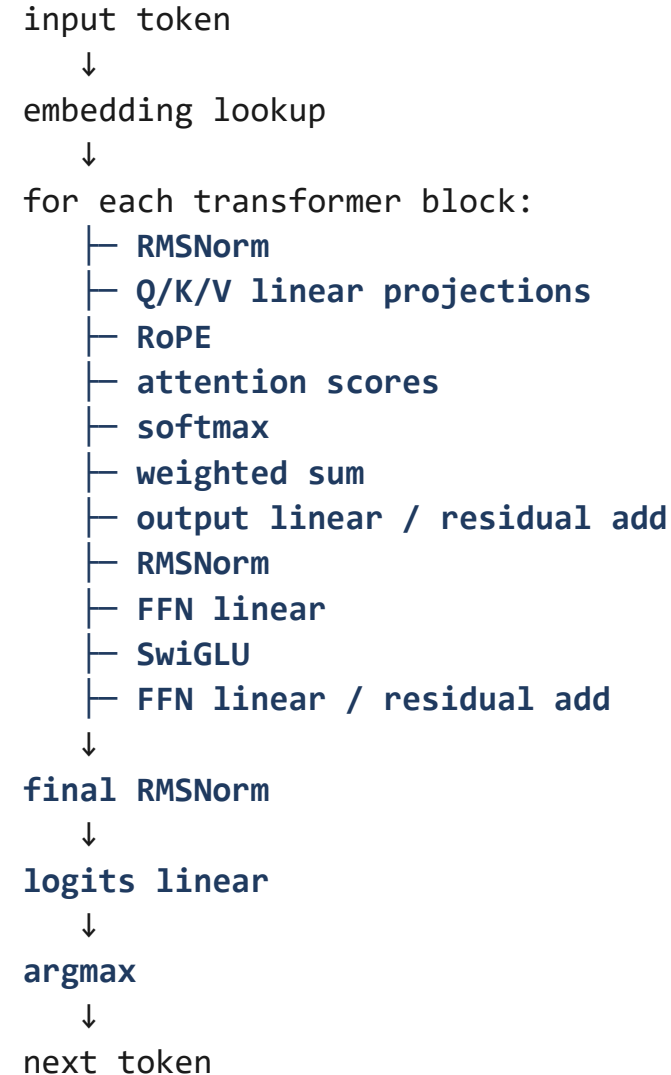
Correctness prototype, not tuned (data held on CPU, not GPU-resident).

What we did

- llama2.c: simple, straight-line code
- factor out the non-deterministic numerical components
- compose different back ends: scalar, NEON, AVX2, AVX-512, CUDA
- hash the I/O values to track bitwise equality across the run

extracted numerical surfaces

- linear / GEMV
- RMSNorm
- attention scores
- softmax
- weighted sum
- RoPE
- SwiGLU
- residual add
- argmax



The Contract

- Fix the expression. Let the execution vary.

The Talk

- Reproducible reduce and scan
- Composable: reproducible parallel algorithms.
- Reproducible Transcendental Math Functions
- Reproducible Linear Algebra.

Floating point is not the reals

When we implement mathematical functions using floating-point numbers, it is easy to confuse them with real numbers.

$$(A + B) + C$$

same real expression

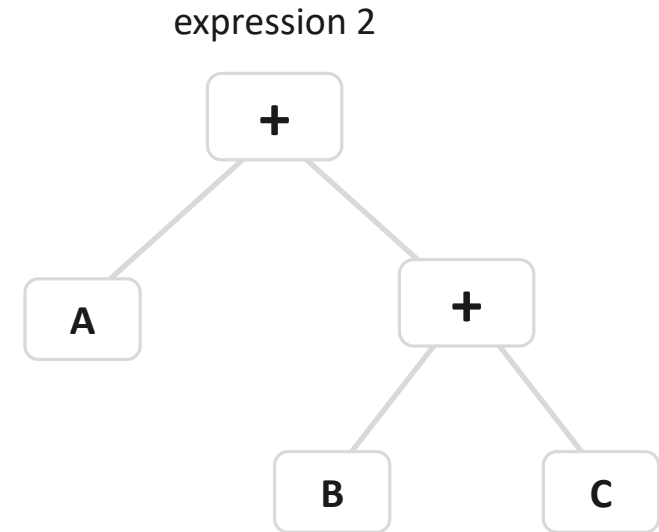
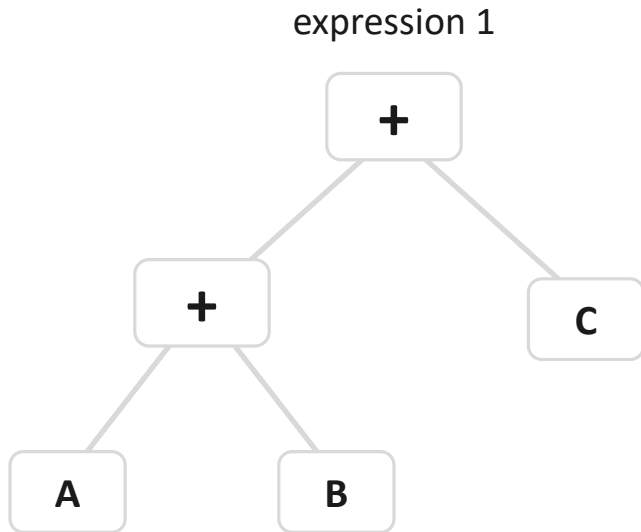
$$A + (B + C)$$

different floating-point expression

The parentheses are part of the computation.

Regrouping changes the computation

If we want exactly the same answer, the expression has to be repeated.



Same letters. Different dependency graph. Different rounded result is allowed.

Modern primitives make regrouping the fast path

reduce and scan are fundamental : They let the implementation regroup work for performance.

vector lanes
SIMD width

threads

GPU
warps / blocks

Each one can choose a different grouping.

Different grouping → different expression → different bits

It's baked in. It's broken.

Two routes to reproducibility

Route one: make regrouping irrelevant.

Associative / exact state

Integer arithmetic is the clean example.

Accumulate exactly, then round at the specified point.

Very strong. Sometimes expensive.

Route two: make regrouping explicit.

Named floating expression

Keep floating point.

Fix which operands meet, and where the observations are.

Same expression, different execution.

We will pay homage to the integer route later. Most of this talk follows the second route.

The expression has to remain constant and known

That is the whole turn.

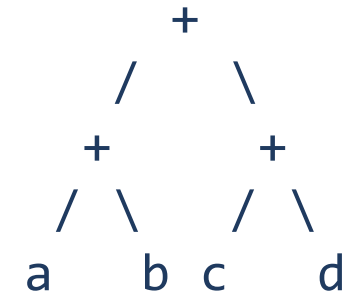
**Fix the expression.
Let the execution vary.**

Not the same schedule. Not necessarily the same instructions.
The same numerical expression.

Expression Geometry

Expression geometry

Expressions aren't just a collection of operators and values connected together.
They have other properties, and those properties can be geometric in nature.



scalability

error behaviour

reproducibility

consistency

streaming

hardware mapping

Property1: Scalability

For real numbers, reshaping preserves the mathematical value.
Parallelism is a broken dependency.

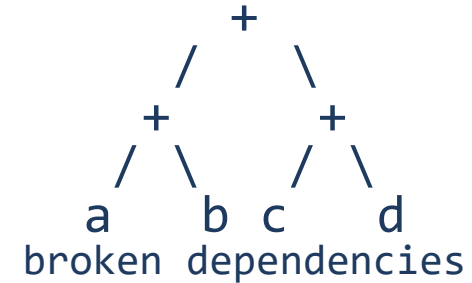
Left fold

$((a + b) + c) + d$

long dependency chain

geometry can be an
implementation
decision

Tree expression



Each broken dependency is an opportunity for parallelism.

Property 2: Error behaviour

left fold

longest path: $O(N)$

rounding occurs along
one long chain

balanced tree

longest path: $O(\log N)$

rounding occurs along
shorter paths

Same data. Same operator. Different geometry. Different error behaviour.

Property 3: Reproducibility

Agree on a unique expression

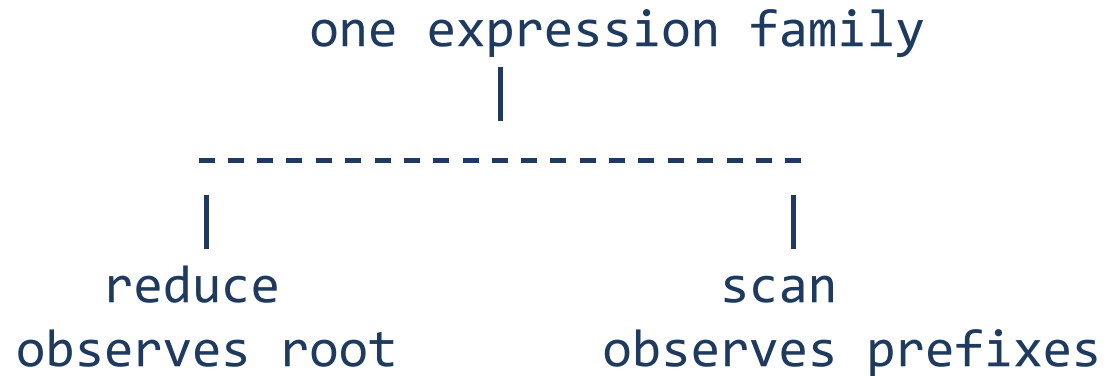
same expression
+ same primitive semantics

same bits

Different SIMD widths, thread counts and machines are realizations — not the contract.

Property 4: Consistency

Different algorithms can observe the same expression.

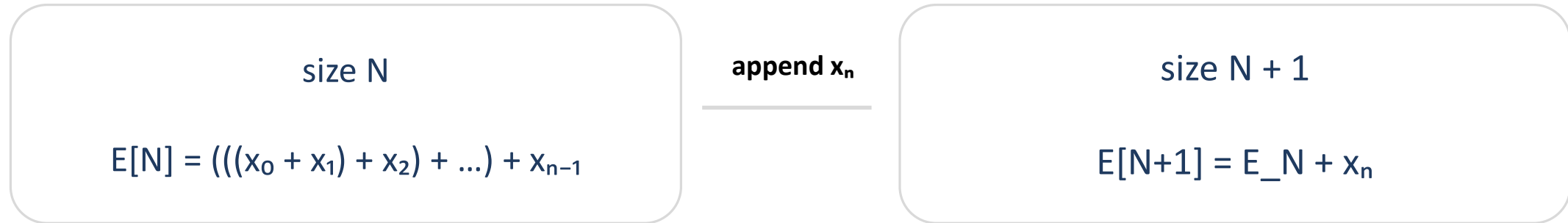


`scan(x).back() == reduce(x)`

probability mass / CDF endpoint → exactly one

Property 5: Prefix persistence

A property of the relationship between the geometry at N and $N+1$.



The old expression survives inside the new one.

Prefix persistence → streaming

Property 6: Structural absence

Absence can belong to the expression geometry.

Padding / identity



forces a property onto the operator

Structural absence



operator is not applied
value passes through

What looked like an operator property becomes a property of the expression geometry.

Property 7: The geometry meets the machine

These properties are not purely abstract.

dependency	→	latency
independent branches	→	SIMD lanes · threads · warps
live subexpressions	→	register pressure
irregular edges	→	masks · tail handling
blocks	→	locality · communication

The expression is logical; the consequences become physical.

Design Question: So which geometry do we want?

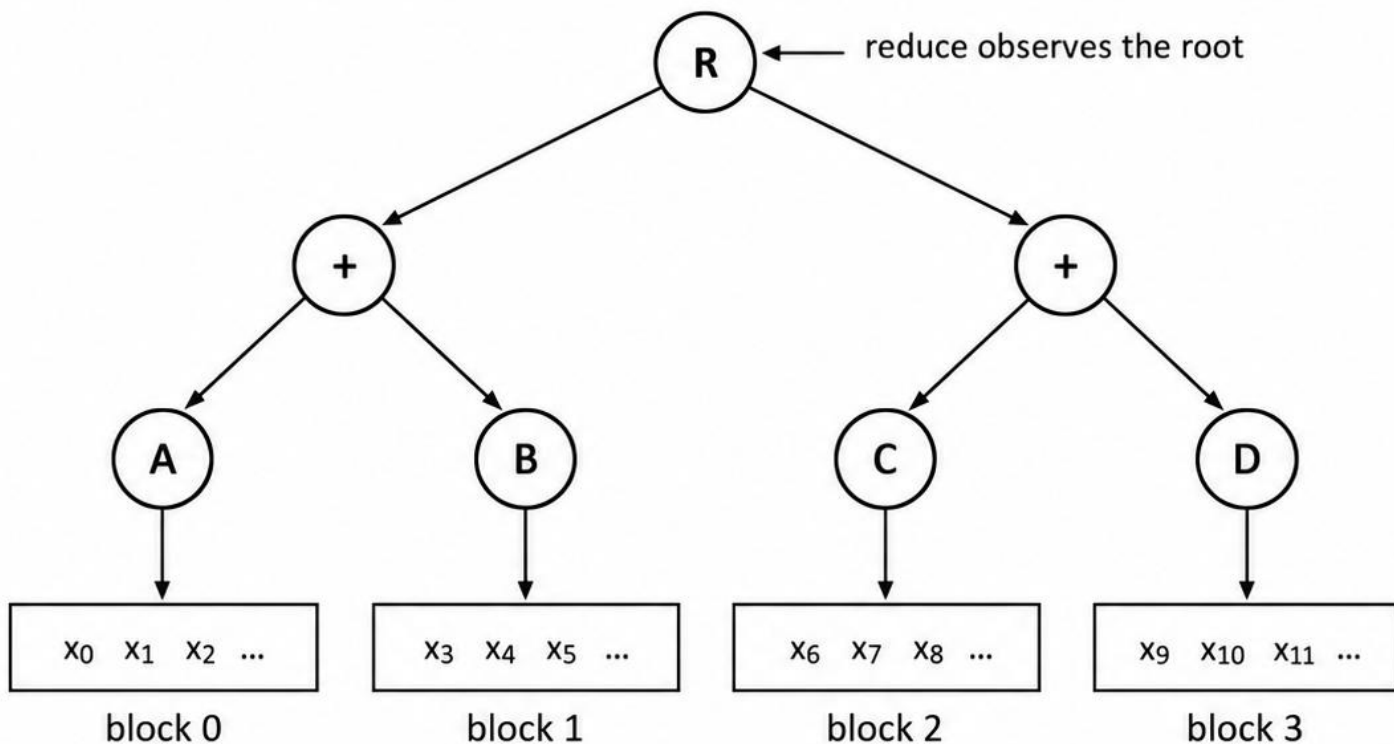
- reproducible
- scalable
- low-error paths
- consistent reduce / scan
- prefix-persistent
- streamable
- machine-friendly
- structural absence

**Choose the geometry
for the properties you need.**

Foundations reduce and scan

START WITH REDUCE

First make the expression explicit



- Start with a balanced blocked expression
- Blocks make SIMD a realization detail
- The root gives the reduce result

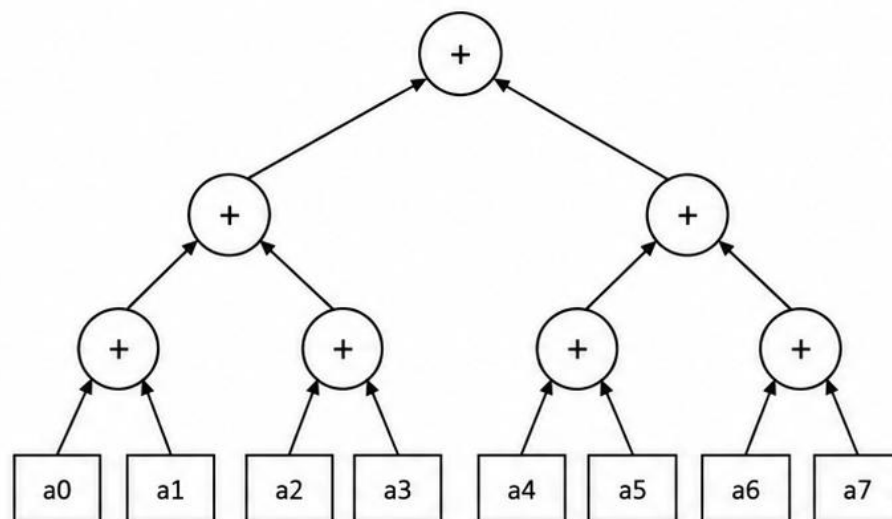
The takeaway:

The first win was to make the expression explicit.

THE BALANCED TREE WAS SLOW

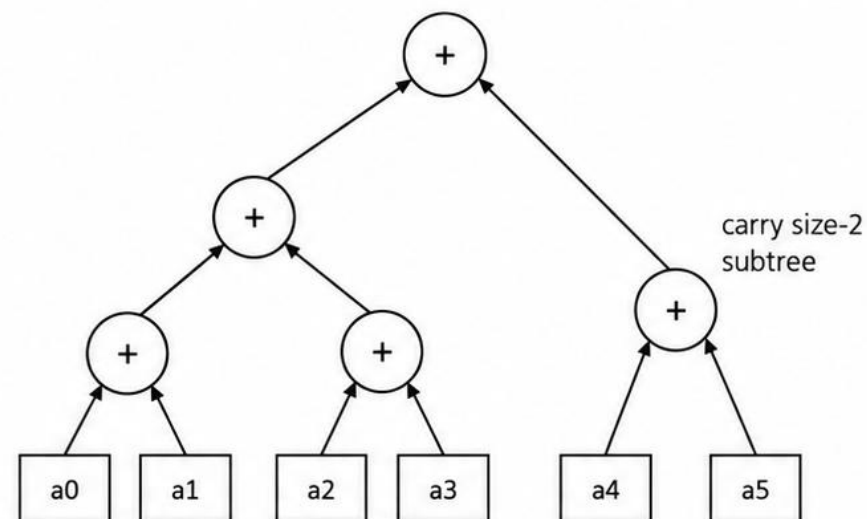
Arbitrary lengths leave an irregular frontier

Power of two: clean full tree



uniform combines

Arbitrary length: ragged frontier



irregular combines / control at the frontier

The mathematical structure was simple

The physical realization was not uniform

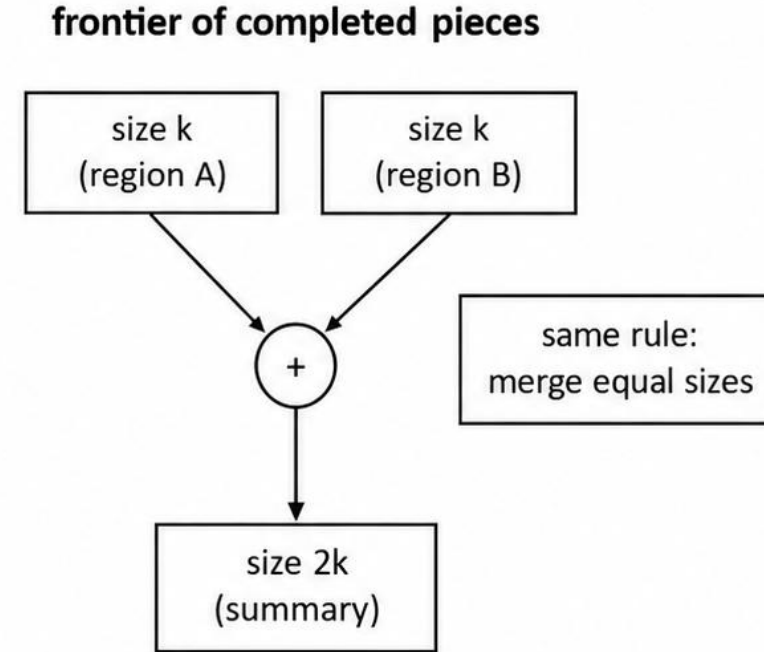
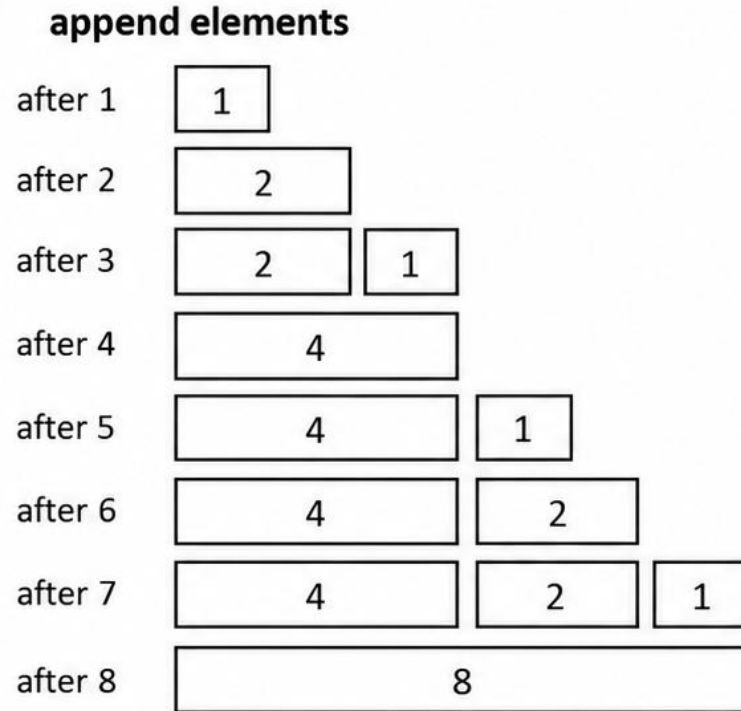
Arbitrary lengths introduced carries and conditionals

The takeaway:

The tree was correct, but not yet friendly to the machine.

DISCOVERY: SHIFT-REDUCE FRONTIER

Completed power-of-two pieces stay completed



Shift-reduce keeps a compact frontier

Completed regions are natural thread work

Appending input does not redraw completed pieces

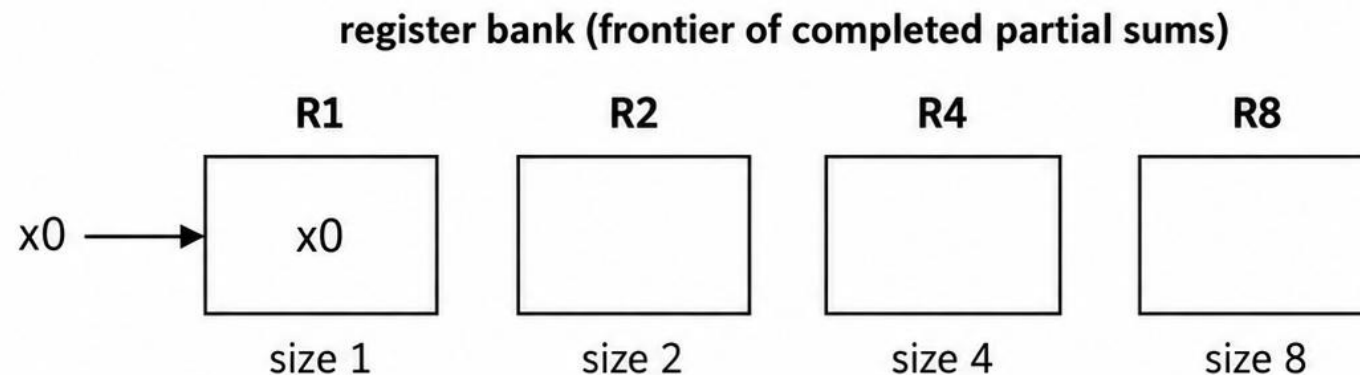
The takeaway:

Prefix persistence appeared
as a useful geometry property.

Shift-Reduce Details

DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x_0 — the first completed partial sum



after 1 element:

$R1 = x_0$

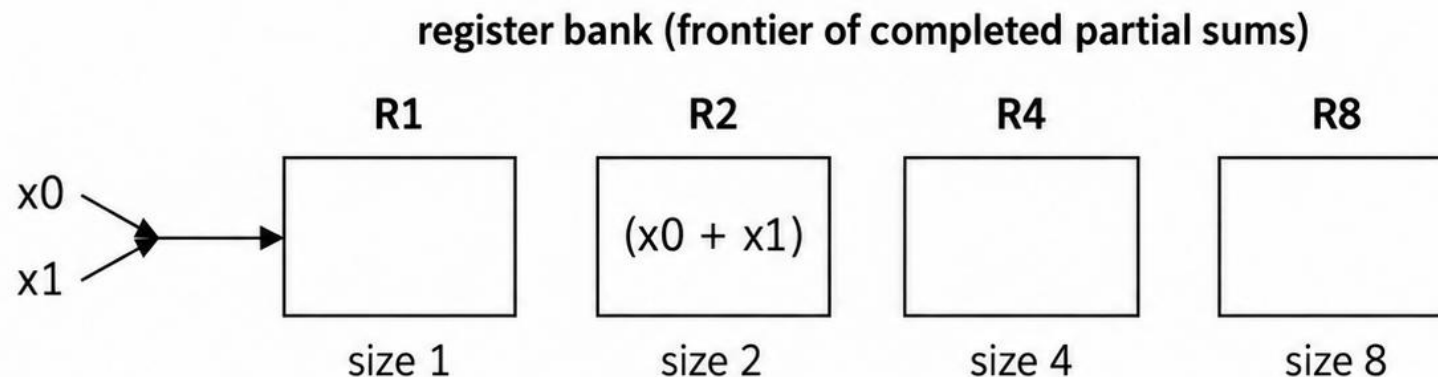
- Add elements one at a time
- Each occupied register stores a completed partial sum
- Register size shows the length of the completed region

The takeaway:

After 1: the frontier contains one size-1 partial sum.

DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x_1 — two size-1 sums carry into R2



after 2 elements:

$$R2 = (x_0 + x_1)$$

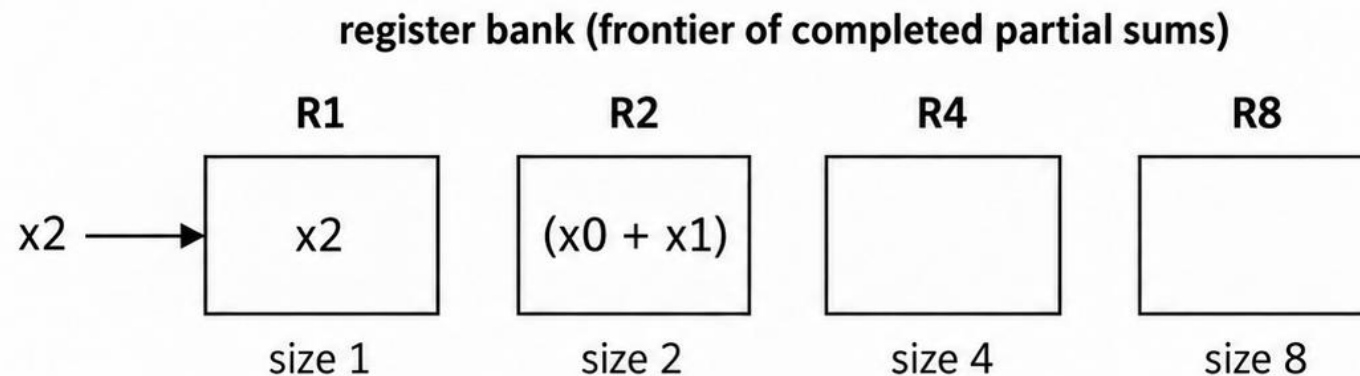
- Two size-1 partial sums merge into one size-2 partial sum
- Equal-size sums carry into the next register
- The frontier stays compact.

The takeaway:

After 2: one size-2 partial sum replaces two size-1 sums.

DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x_2 — the frontier becomes $R_2 + R_1$



after 3 elements:

$R_2 = (x_0 + x_1)$
 $R_1 = x_2$

- Completed partial sums remain in place
- Occupied registers correspond to $3 = 2 + 1$
- The expression grows without redrawing completed pieces.

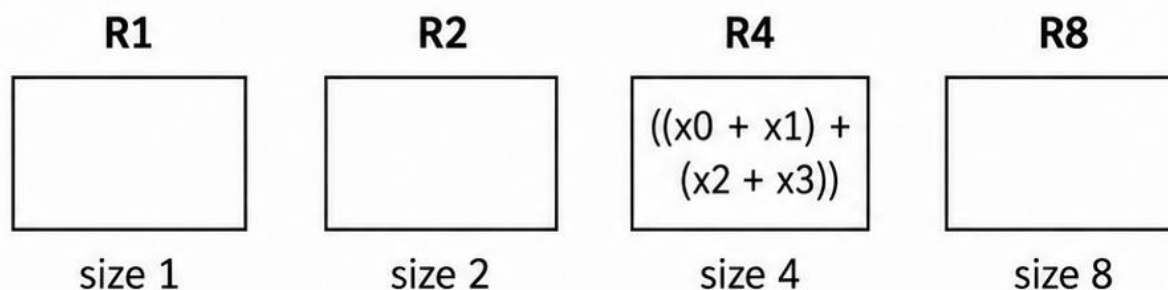
The takeaway:

After 3: the frontier holds one size-2 sum and one size-1 sum.

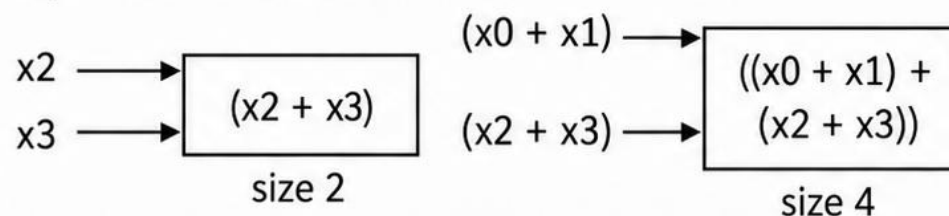
DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x_3 — a carry chain produces R_4

register bank (frontier of completed partial sums)



carry chain for x_2, x_3 :



after 4 elements:

$$R_4 = ((x_0 + x_1) + (x_2 + x_3))$$

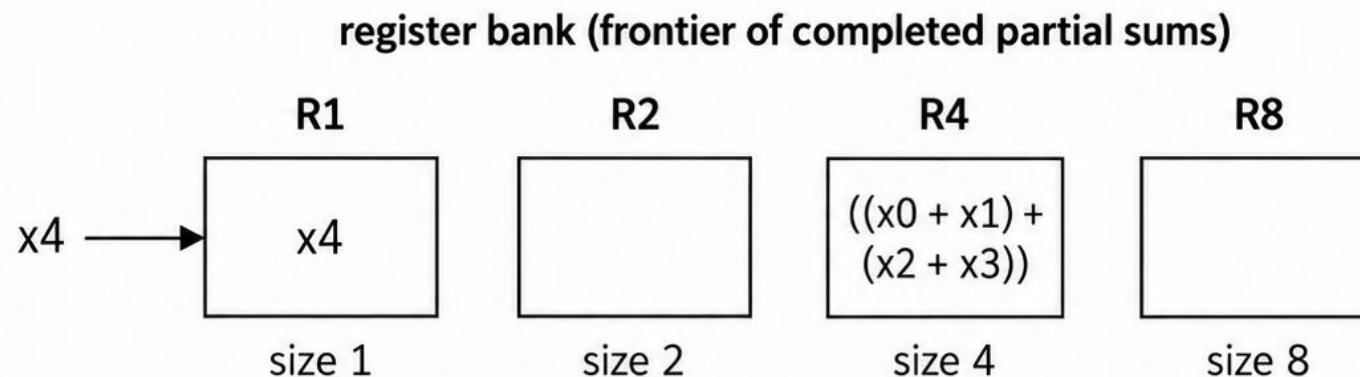
- x_2 and x_3 first form a size-2 partial sum
- Two size-2 sums then carry into R_4
- Prefix length 4 is represented by one occupied register.

The takeaway:

After 4: the first size-4 partial sum appears.

DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x_4 — the frontier becomes $R_4 + R_1$



after 5 elements:

$$R_4 = ((x_0 + x_1) + (x_2 + x_3))$$
$$R_1 = x_4$$

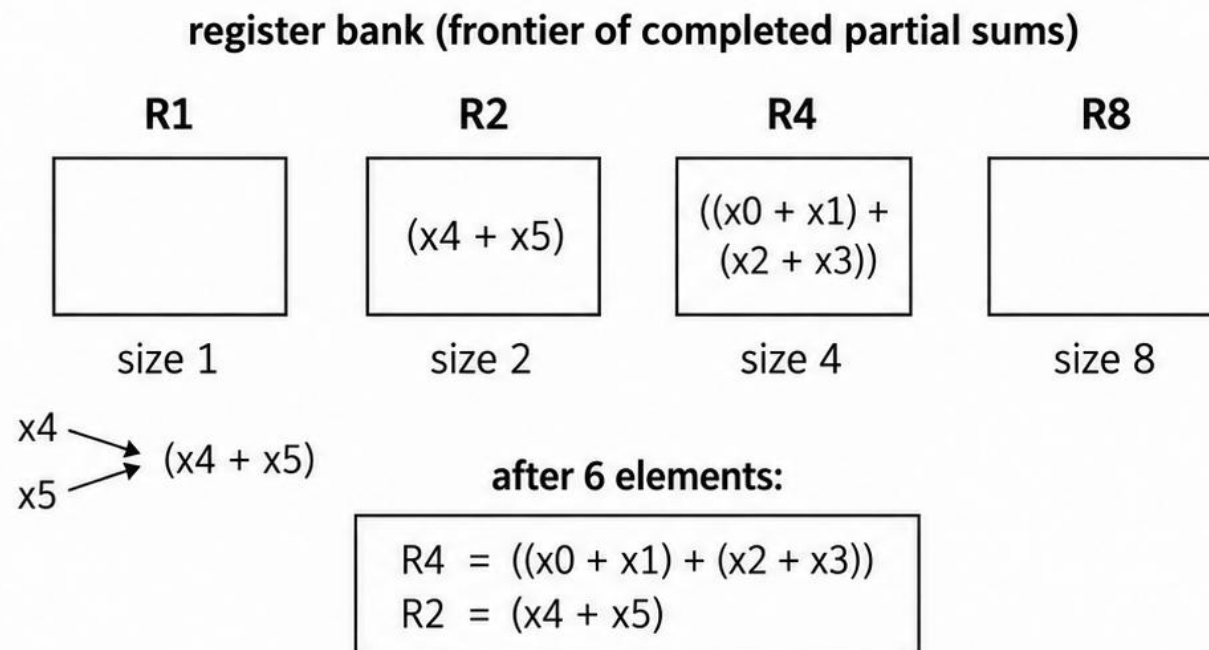
- The completed size-4 region remains completed
- The new element occupies the size-1 register
- Occupied registers correspond to $5 = 4 + 1$.

The takeaway:

After 5: the frontier holds a size-4 sum and a size-1 sum.

DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x_5 — the frontier becomes $R_4 + R_2$



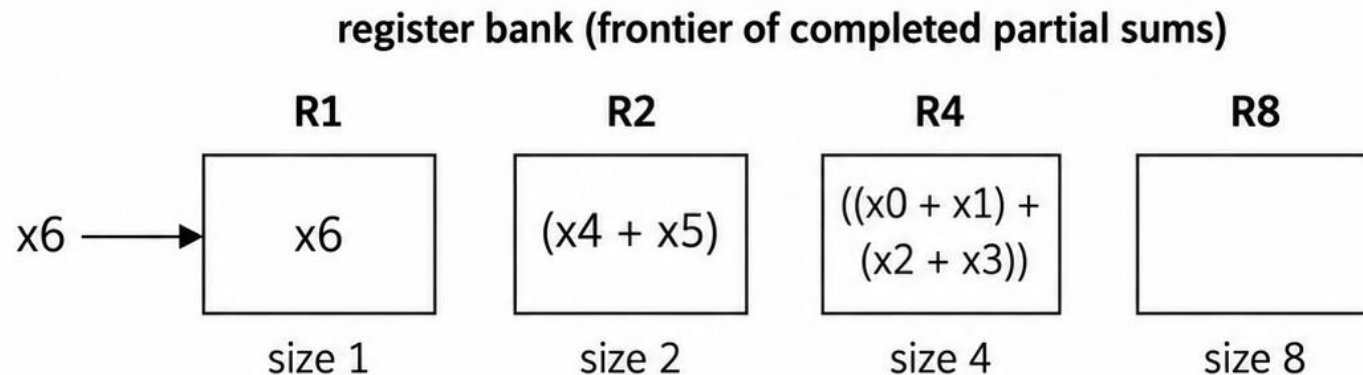
- x_4 and x_5 combine into a size-2 partial sum
- The size-4 partial sum stays unchanged
- Occupied registers correspond to $6 = 4 + 2$.

The takeaway:

After 6: the frontier holds a size-4 sum and a size-2 sum.

DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x6 — the frontier becomes $R4 + R2 + R1$



after 7 elements:

$R4 = ((x0 + x1) + (x2 + x3))$
$R2 = (x4 + x5)$
$R1 = x6$

- Completed pieces remain completed
- Occupied registers correspond to $7 = 4 + 2 + 1$
- The frontier stores only $O(\log N)$ partial sums.

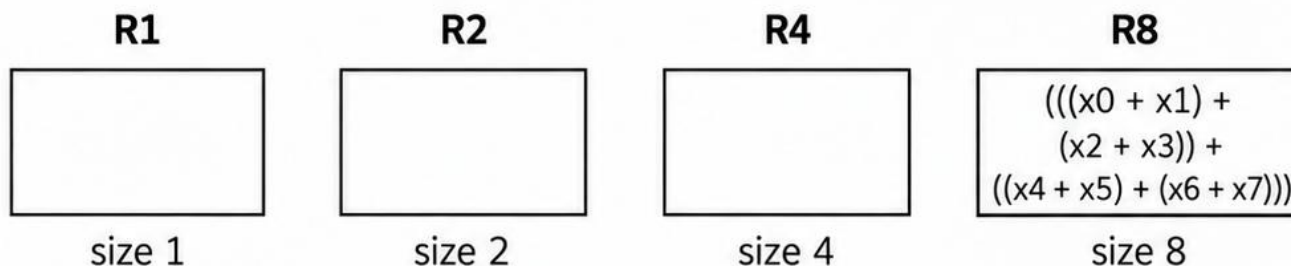
The takeaway:

After 7: the frontier holds size 4, size 2, and size 1 partial sums.

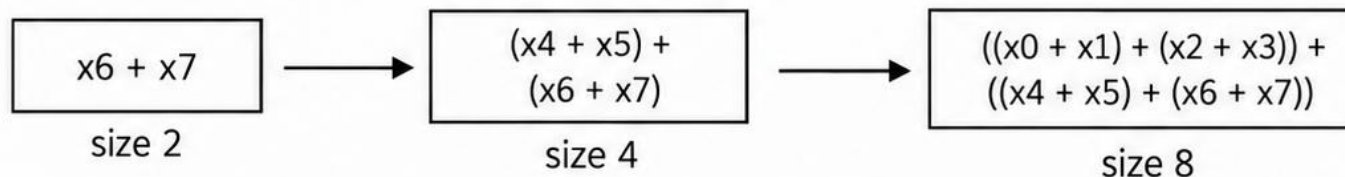
DISCOVERY: A FRONTIER OF PARTIAL SUMS

Append x7 — repeated carries collapse the frontier to R8

register bank (frontier of completed partial sums)



carry chain when appending x7:



after 8 elements:

$$R8 = (((x0 + x1) + (x2 + x3)) + ((x4 + x5) + (x6 + x7)))$$

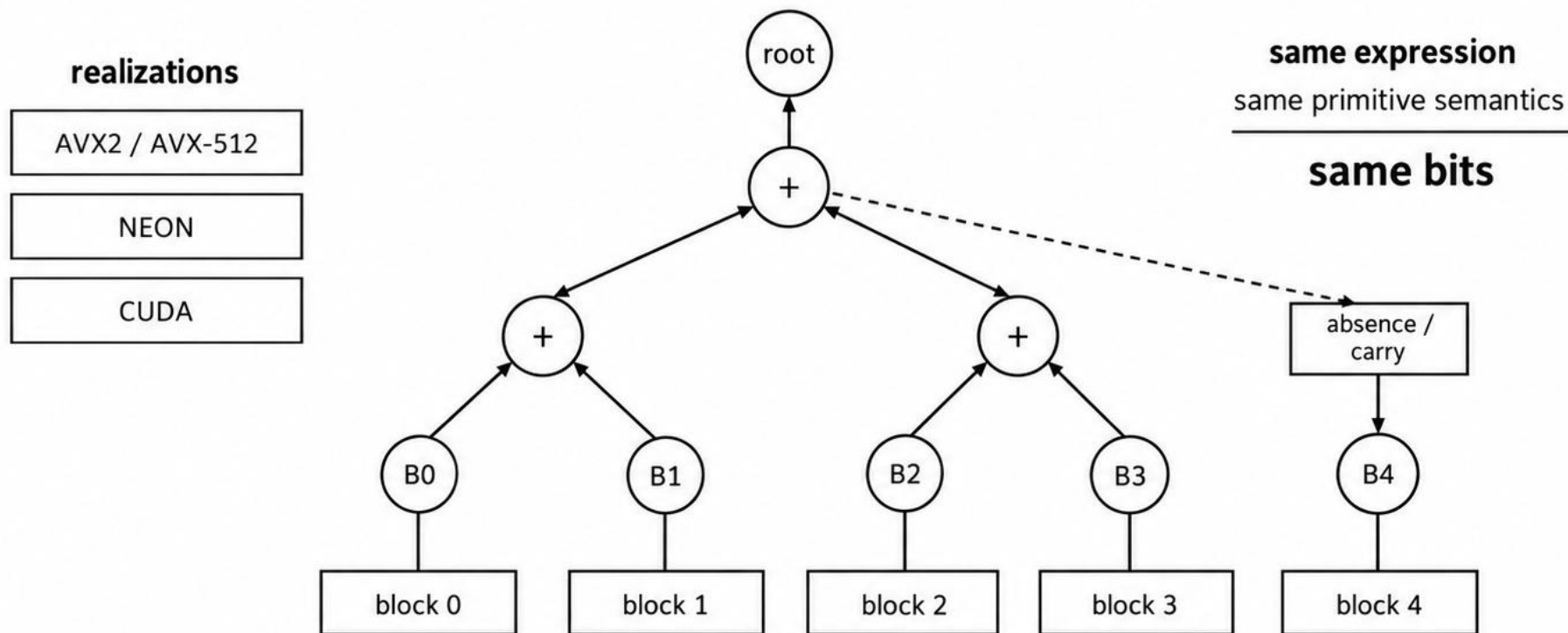
- Appending x7 triggers a carry chain through 1, 2, and 4
- The frontier collapses to one size-8 partial sum
- Occupied registers correspond to $8 = 8$

The takeaway:

After 8: one size-8 partial sum represents the whole prefix.

CANONICAL BLOCKED DYADIC REDUCE

The expression became the contract



- Canonical dyadic structure above blocks
- Blocks choose a machine-friendly realization
- Structural absence handles tails without changing the operator
- Reduce observes the root of the expression

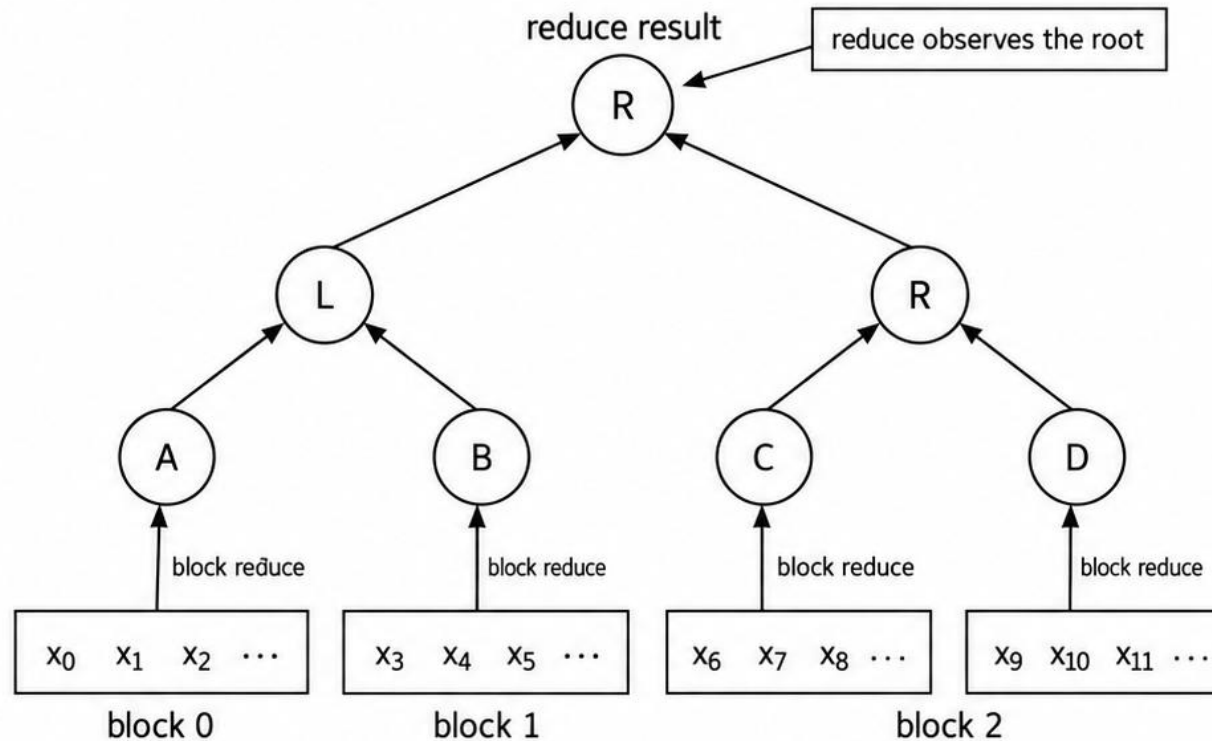
The takeaway:

This is where reduce becomes an Expression Geometry result.

Scan

FROM REDUCE TO SCAN

Start from the successful reduce geometry



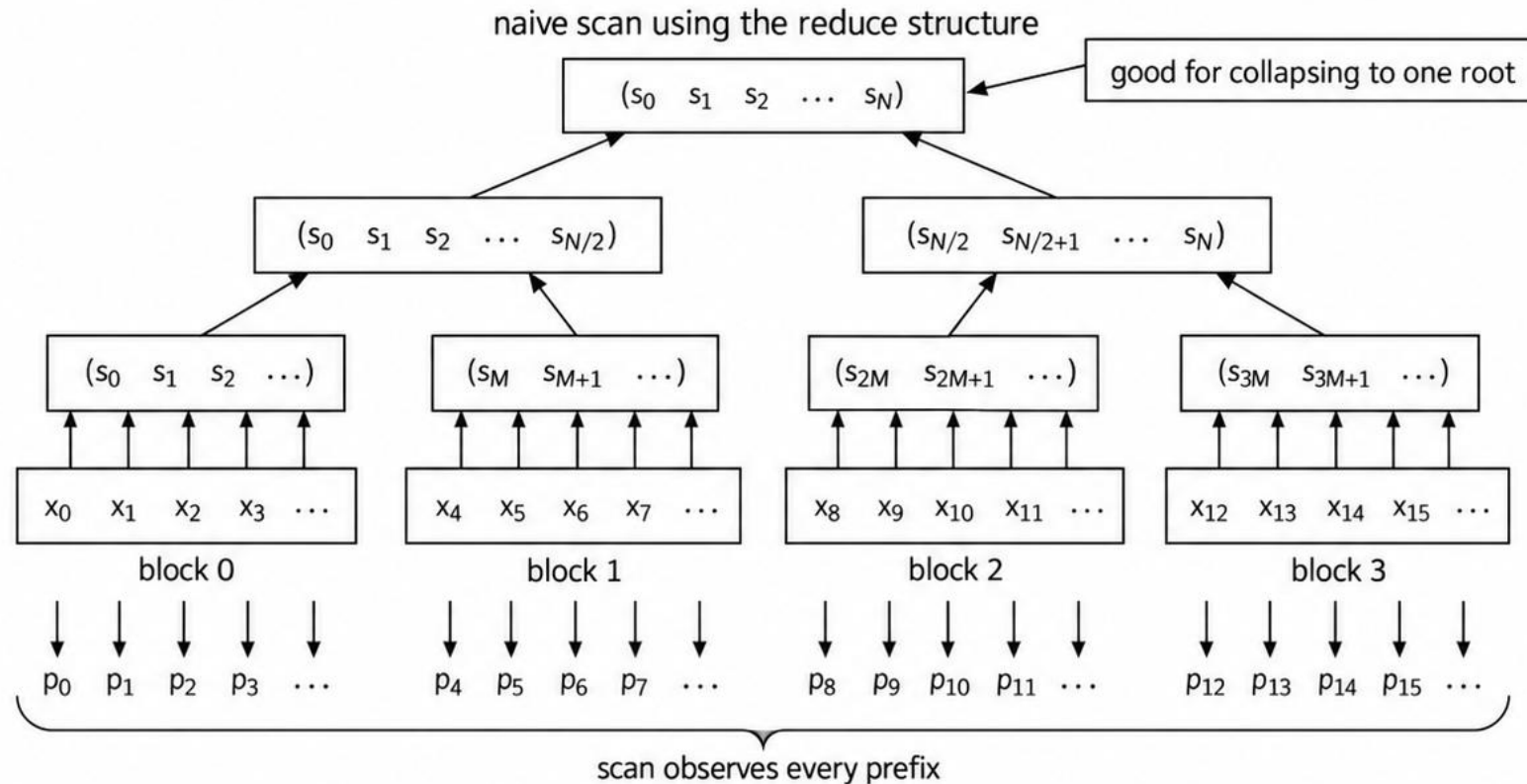
- Balanced / dyadic expression
- Scalable and reproducible
- Performance comparable with existing reduce implementations

The takeaway:

The reduce tree looked like a wonder tree.

THE PROBLEM

Why the reduce tree is the wrong physical shape for scan



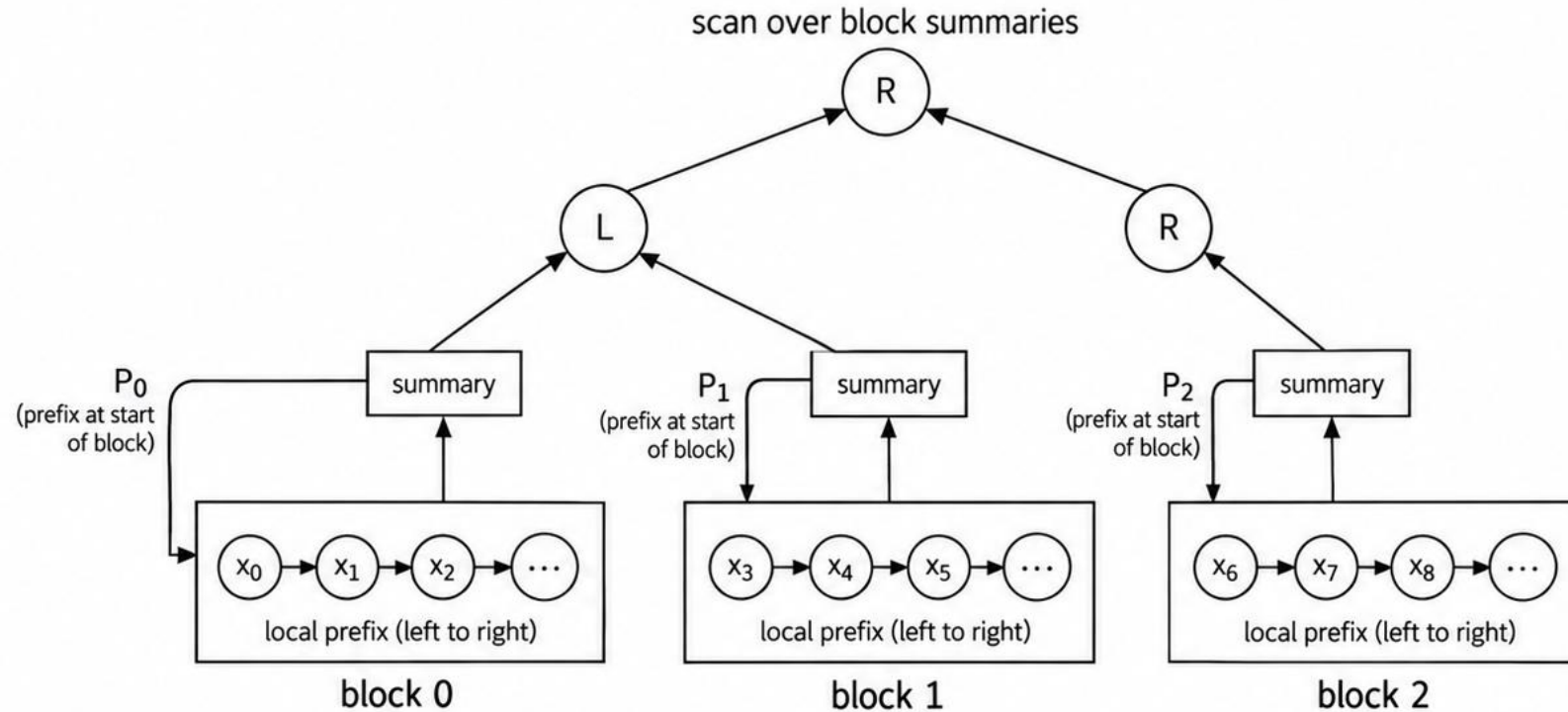
- Reduce reads the input and returns one value
- Scan must return every prefix
- Large block state moving through the upper tree is expensive
- We tried it. It was really, really slow

The takeaway:

Same expression family,
different observation requirements.

THE REDESIGN

Compact information globally, dense work locally



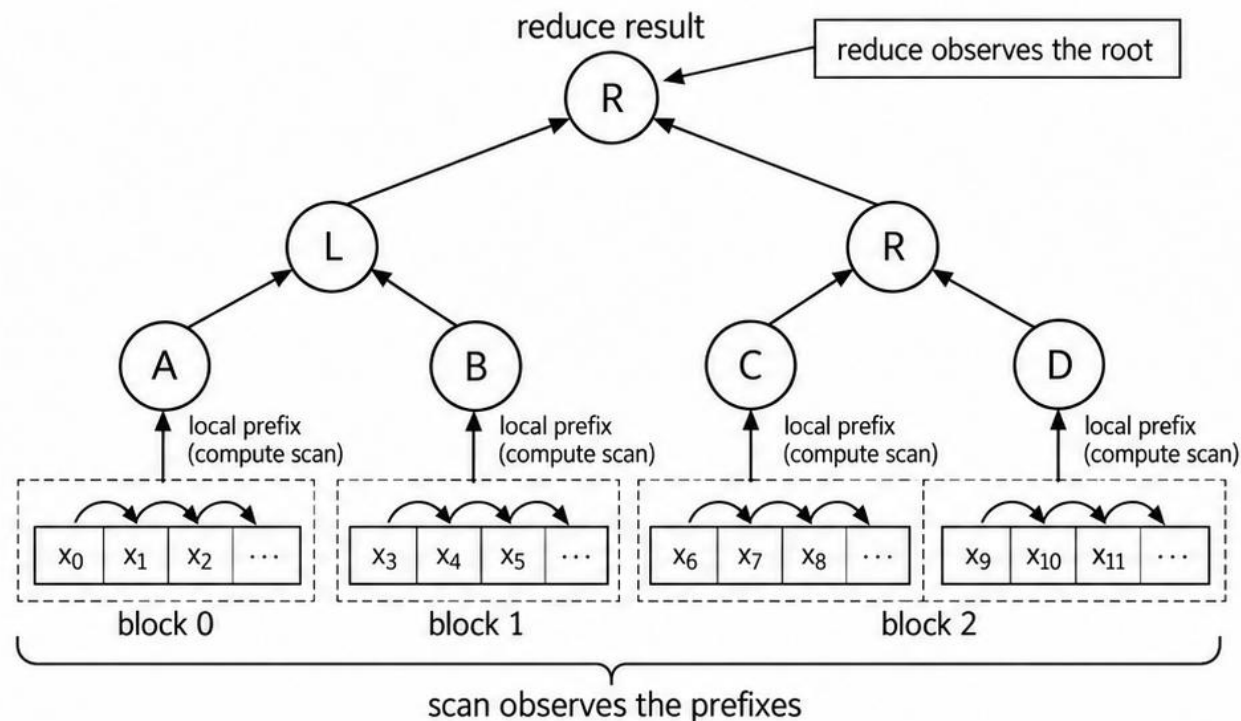
- Each completed block emits a scalar summary
- Upper scan computes the prefix state for each block
- Each block reuses that prefix state to finish its own prefixes locally

The takeaway:

Scan naturally becomes two-level.

ONE EXPRESSION, TWO ALGORITHMS

Scan observes prefixes. Reduce observes the root.



$$\text{scan}(x).\text{back}() == \text{reduce}(x)$$

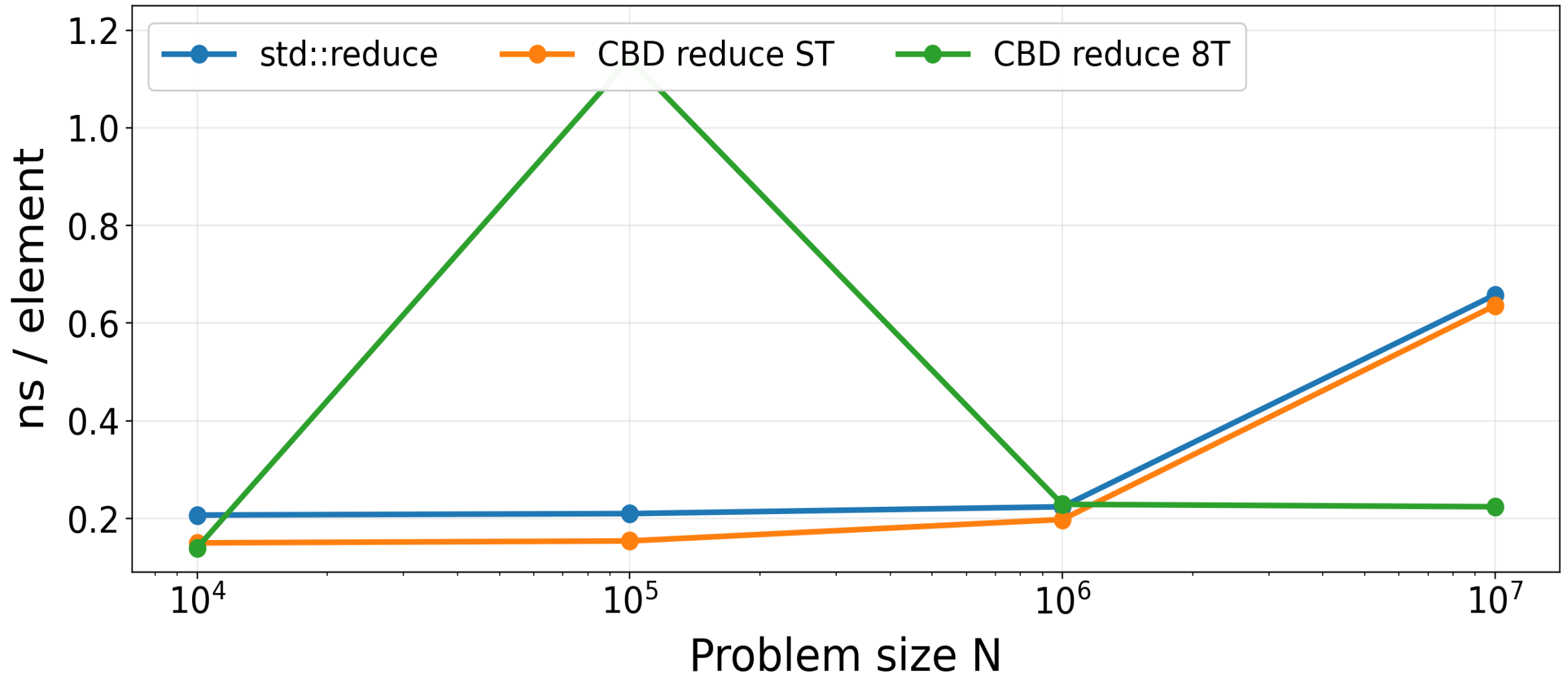
- Reproducibility comes from the expression
- The LCM / SIMD width was a realization decision
- Different machines faithfully realize the same expression
- Consistency: the final scan value exactly matches the reduce

The takeaway:

We stopped forcing scan into reduce.
We designed one expression that supports both.

Canonical blocked-dyadic reduce

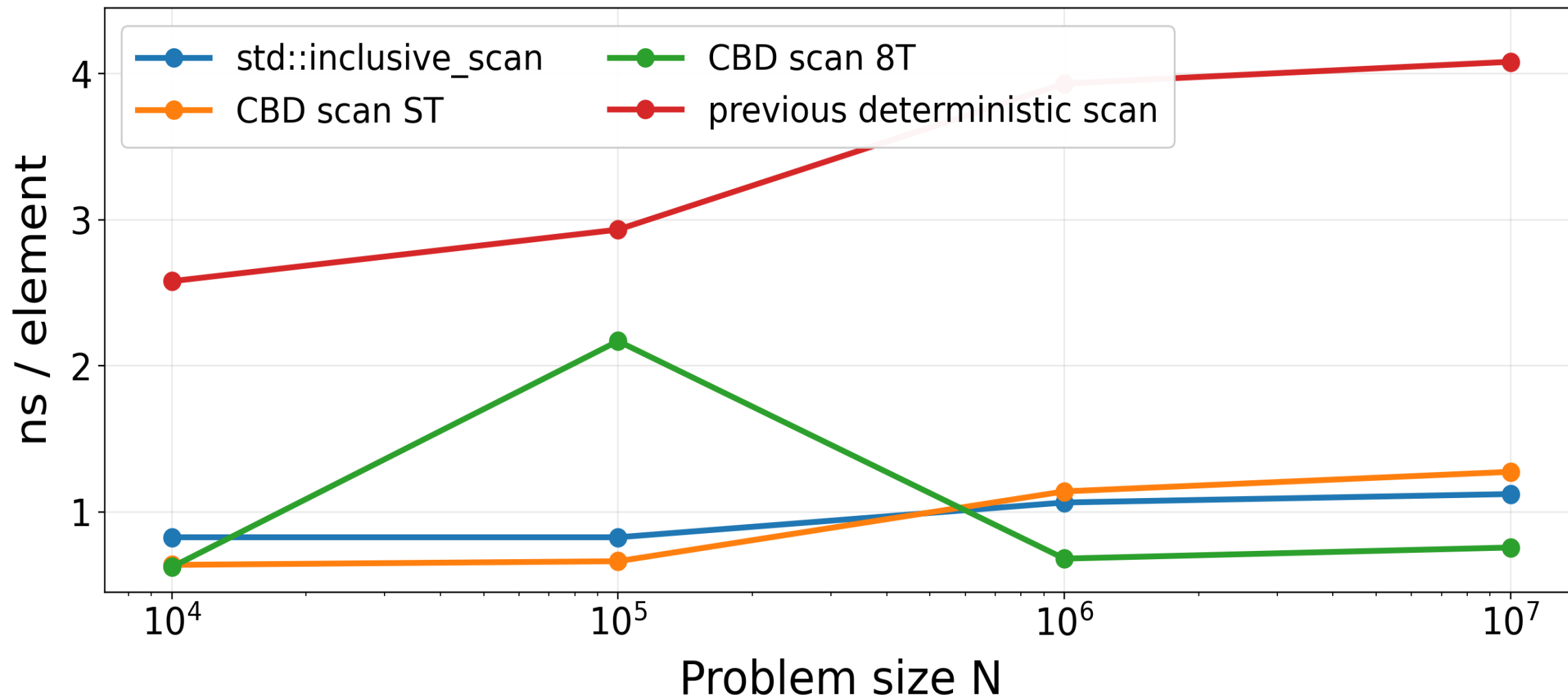
B=64 · uniform random doubles in [-100, 100] · lower is better



Single-thread CBD reduce beats std::reduce across the tested sizes; 8 threads add throughput at large N.

Canonical blocked-dyadic scan

$B=64$ · same expression family as reduce · $\text{scan}[i] == \text{reduce}(X[0..i+1])$ byte-for-byte



The block structure makes scan practical while preserving matched reduce/scan observations.

Can We Compose From Deterministic Primitives ?

From reduce and scan to composition

Deterministic reduce: done. Deterministic scan: done.

The deeper reason for wanting scan: since Blelloch (early 1990s), parallel algorithms have been built from a small set of primitives. Transform, scan, reduce.

Transform is easy: little interaction between elements.

So the question this section answers:

if scan and reduce are deterministic, are the algorithms composed from them deterministic through composition?

The vocabulary: composers and leaf expressions

transform (leaf)

scan

reduce

scatter / gather

Not invented here: candidates from a literature search of existing work and libraries, a small subset pulled out to see what works.

One classification that earned its keep: **leaf expressions** sit at the edges of the expression, at terminal nodes and sources. They do not introduce interactions between elements, so they cannot disturb the interaction geometry. Unary transform is the natural example.

Primitives Decomposition

Building deterministic algorithms from primitives and composers

1 · Leaf helpers



zip



predicate



project



select



adjacent



key / index



2 · Core primitives

$f(x)$

transform



gather



scatter



reduce



scan



3 · Derived blocks



flags



ranks



histogram



segmentation



prefix
allocation



pack /
compact



stable
routing



4 · Composite algorithms

pack / copy_if

stable partition

radix sort

run-length encode

reduce-by-key

group-by

radix select / top-k

segmented pack

sparse assembly

CSR transpose

CSR SpMV

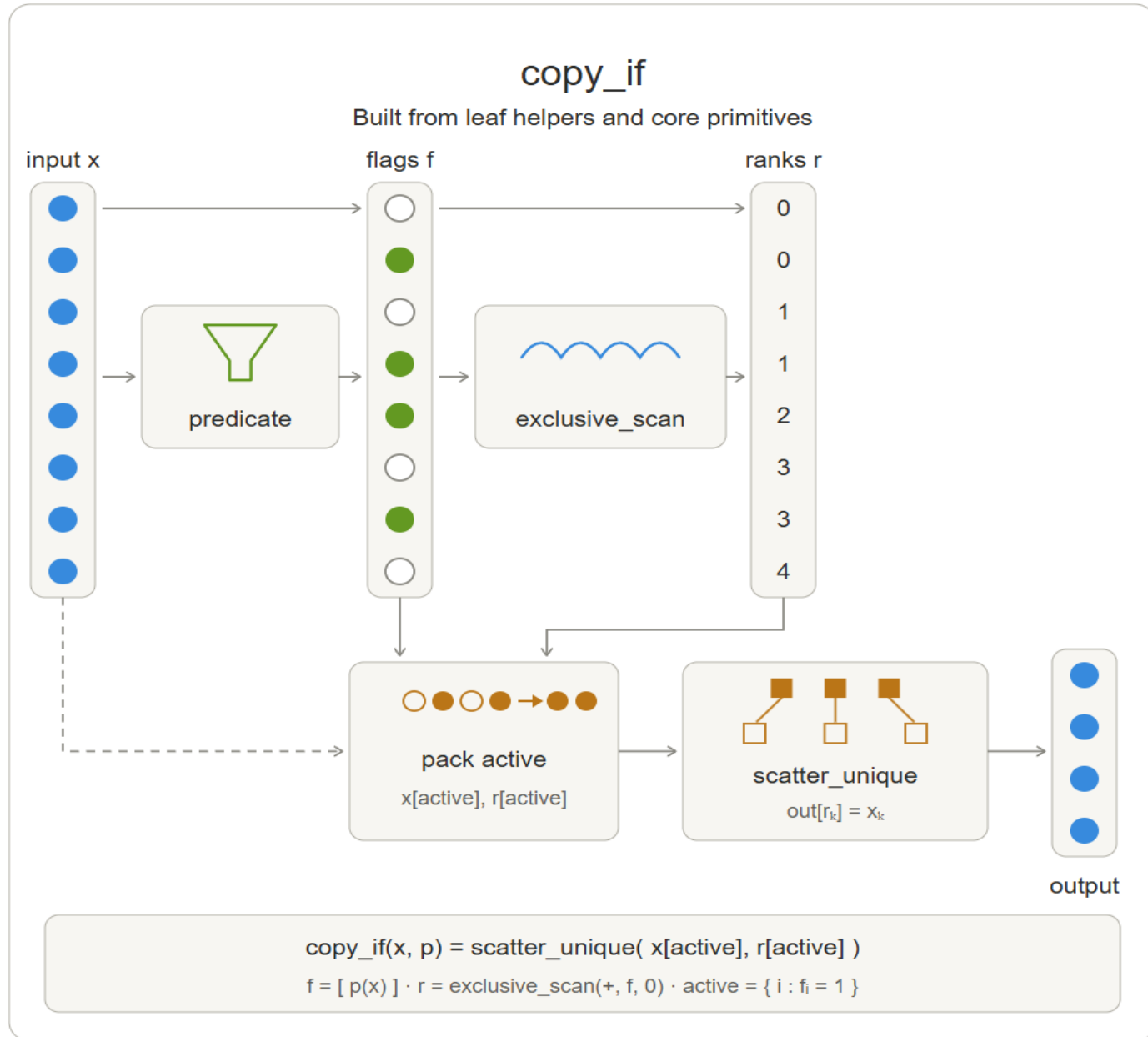
BFS frontier

softmax

segmented softmax

and more ...

Building a new algorithm from the parts



The experiment

- 1 Do they actually compose?
- 2 Which composers do we really need?
- 3 What properties must a composer preserve?
- 4 Does the composition remain deterministic?
- 5 Is it still performant? Does it still scale?

Method: start with simple algorithms, work up to harder ones. Integers first: no floating-point variability, a clean oracle, and direct comparison against other libraries' implementations of the same algorithms. Then vary the physical realization.

The algorithm zoo

Algorithm	Built from
copy_if	flags, ranks (scan), scatter
stable_partition	the same routing, two-sided
sort_u32	histogram, prefix, 4 stable scatter passes
run_length_encode	head flags, scan, gather
reduce_by_key	head flags, scan, one canonical reduce per group
group_by	stable sort by key, then reduce_by_key

Routing does the moving; positions are fully specified.

The arithmetic lives in exactly one place: the canonical reduce. That is the only point where the expression contract is needed.

Each composition is checked bit-identical against the sequential observation before it is timed.

Varying the realization

SIMD width

scalar, AVX2, AVX-512, NEON via the compiler

Threads

1 to 8 workers, including odd counts

Platform

Apple M4 (NEON), x86 (Intel and AMD), NVIDIA CUDA

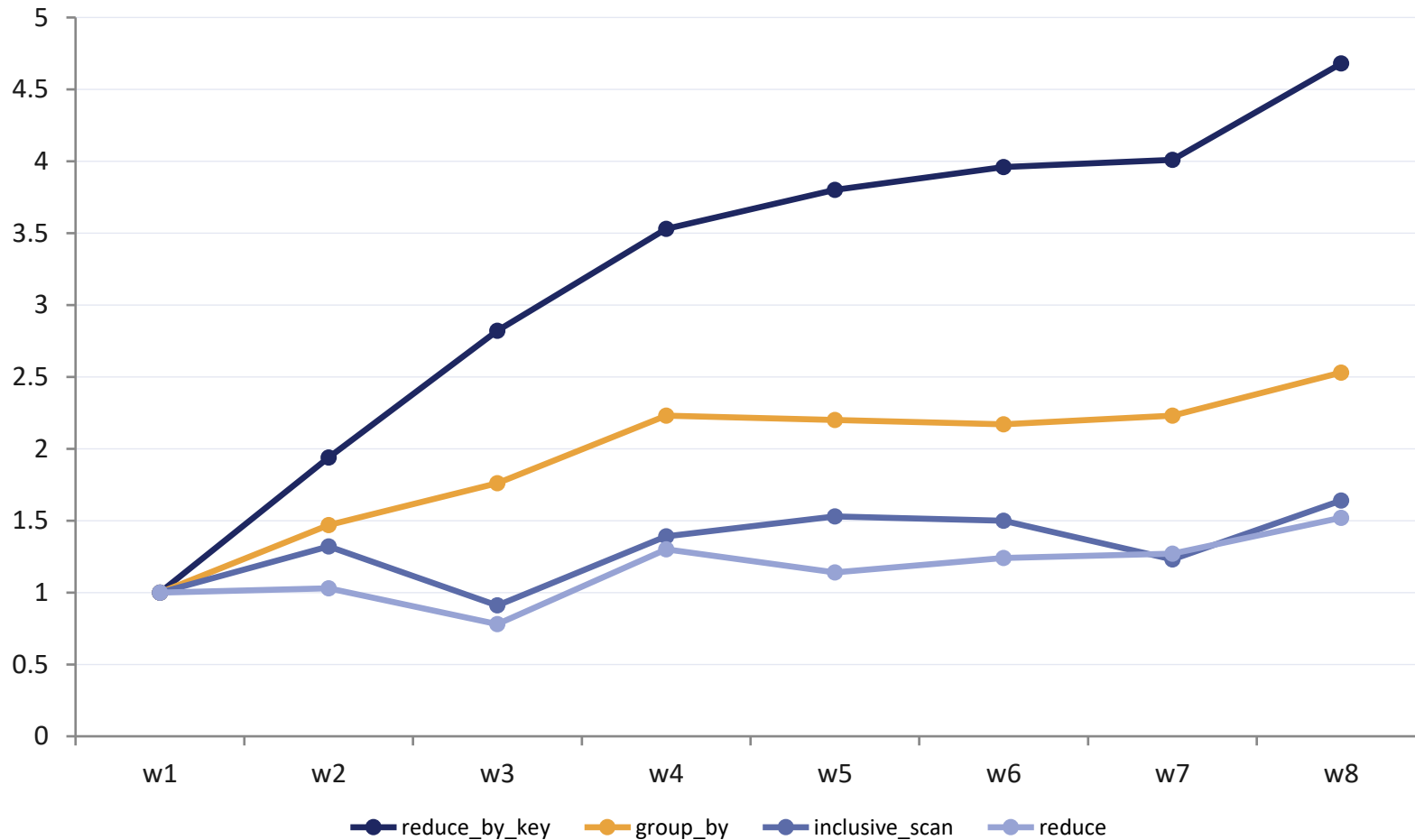
Compiler

GCC, Clang, MSVC, Apple clang, nvcc

Same algorithmic construction, every realization: **the same result, to the bit**. Sixteen cases, two corpus sizes, one hash table shared by all of the above.

Apple M4: throughput as threads increase

fp64 hostile corpus, $n = 2^{22}$, speedup vs one worker



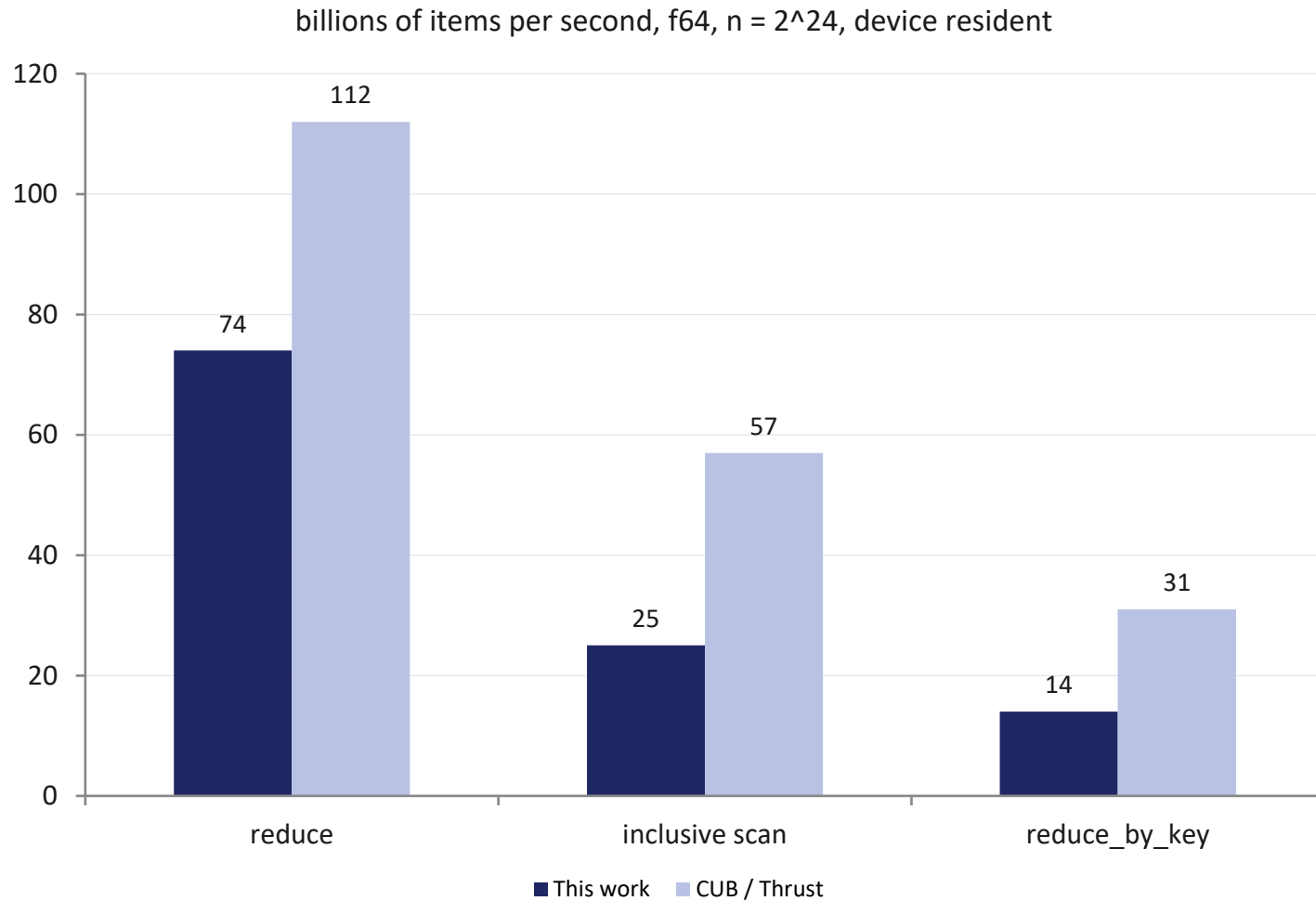
Sensitivity to thread count varies, and weak scaling is not failure to parallelise.

reduce and scan sit near the memory-bandwidth limit with few threads; more workers buy little.

reduce_by_key and group_by do more work per element, so they have more headroom: 4.7x and 2.5x.

Output hash: identical at every point.

NVIDIA RTX 4090



Reduce: about 74 against 112 billion items per second.

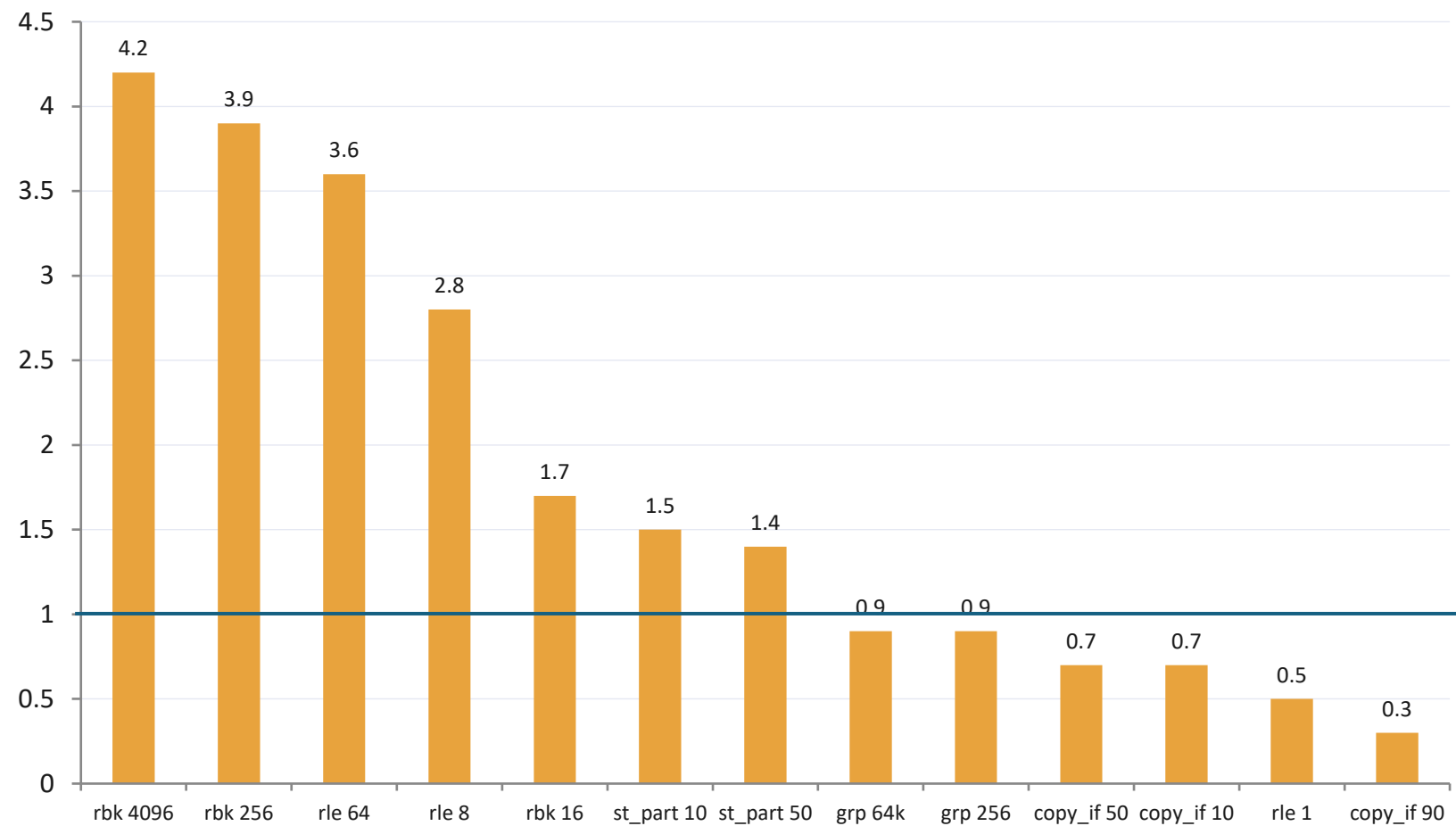
Scan: somewhat under half of CUB's throughput. `reduce_by_key` also behind.

NVIDIA are quite proud of their scan technology, with good reason. This is not an easy baseline to take on.

The same construction, bit-identical to the CPU, running on their silicon at this distance from their best.

Intel oneDPL, dual Xeon Silver, 8 workers

u32, n = 2^20, Intel's prescribed build both sides



A rather different story. Without an enormous amount of optimisation effort, most of these composites perform at least as well as the Intel package.

copy_if is the notable exception, and that operation has probably been optimised to death.

The physical machine pushing back: memory hierarchy, cache behaviour and the expression geometry interact.

What makes composition work

The answer is almost embarrassingly simple.

A composer has to respect the invariants required to propagate the properties being preserved.

In other words: it is a good composer provided it does not break the things we require to remain true.

For determinism, the invariants are concrete: positions fully specified in every routing step, stability where order carries meaning, and every combined value flowing through the one contracted reduce. Leaf expressions cannot break anything, which is why the classification pays for itself.

Result

Yes: algorithms compose from these primitives into a zoo of deterministic parallel algorithms.

They exploit SIMD. They scale across threads. They run on Apple, Intel, AMD and NVIDIA hardware.

Vary the SIMD width, vary the thread count, vary the platform: the same result, which was the point of the experiment.

And the performance results are not discouraging.

Soft Max

1 max --> 2 exp --> 3 retain --> 4 sum --> 5 normalize

SOFTMAX: CONTRACTS COMPOSE

DEFINITION

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad i = 1, \dots, n$$

SHIFTED FORM, EVALUATED HERE

$$y_i = \frac{e^{z_i - m}}{\sum_{j=1}^n e^{z_j - m}}, \quad m = \max_{1 \leq k \leq n} z_k$$

Identical in exact arithmetic.

SOFTMAX: CONTRACTS COMPOSE

One algorithm. Several numerical contracts. C++ composition used in reproducible llama2.c.

1 max --> 2 exp --> 3 retain --> 4 sum --> 5 normalize

1 MAXIMUM

$$m = \max_i z_i$$

2 SHIFTED EXP

$$e_i = \exp(z_i - m)$$

3 RETAIN e_i

$$\mathbf{e} = (e_1, \dots, e_n)$$

4 NORMALIZATION SUM

$$s = \sum_{i=1}^n e_i$$

5 NORMALIZE

$$y_i = e_i / s$$

SOFTMAX: CONTRACTS COMPOSE

1 max --> 2 exp --> 3 retain --> 4 sum --> 5 normalize

1 MAXIMUM

$$m = \max_{1 \leq i \leq n} z_i$$

aggregation expression + observation

C++ COMPOSITION

```
auto maximum = softmax_max_f32_unchecked(input);
```

```
canonical_block_dyadic_reduce<64>( left_biased_max_f32{},  
                                     input, -CM_INF);
```

REALIZATION / OBSERVATION

Named expression: canonical_block_dyadic<64>

Observation: reduce root

lambda is left-biased max

SOFTMAX: CONTRACTS COMPOSE

1 max --> 2 **exp** --> 3 retain --> 4 sum --> 5 normalize

2 SHIFTED EXP

$$e_i = \exp(z_i - m)$$

contracted elementwise result

C++ COMPOSITION

```
kernel_transform(input_view, output_view,  
                 shifted_exp_f32{maximum});
```

```
float operator()(float x) { return cm_exp(x - maximum); }
```

REALIZATION / OBSERVATION

Block / scalar transform callable

Contracted FP32 cm_exp

Same operation over block<float, W>

SOFTMAX: CONTRACTS COMPOSE

1 max --> 2 exp --> **3 retain** --> 4 sum --> 5 normalize

3 RETAIN e_i

$$\mathbf{e} = (e_1, e_2, \dots, e_n)$$

exponentials[] become the next input

C++ COMPOSITION

```
std::span<const float> exponentials{ output.data(),  
                                     output.size() };
```

REALIZATION / OBSERVATION

Controlled materialization

Not claimed as one-pass softmax

SOFTMAX: CONTRACTS COMPOSE

1 max --> 2 exp --> 3 retain --> 4 **sum** --> 5 normalize

4 NORMALIZATION SUM

$$s = \sum_{i=1}^n e_i$$

aggregation expression + observation

C++ COMPOSITION

```
auto sum = softmax_sum_exp_f32_unchecked(exponentials);
```

```
canonical_block_dyadic_reduce<64>( std::plus<float>{},  
                                     exponentials, 0.0f);
```

REALIZATION / OBSERVATION

Named expression: canonical_block_dyadic<64>

Observation: reduce root

Geometry is specified, not inferred

SOFTMAX: CONTRACTS COMPOSE

1 max --> 2 exp --> 3 retain --> 4 sum --> **5 normalize**

5 NORMALIZE

$$y_i = \frac{e_i}{s}$$

IEEE primitive operation

C++ COMPOSITION

```
kernel_transform(exponentials_view,  
                 output_view,  
                 detail::divide_by_sum_f32{sum});
```

REALIZATION / OBSERVATION

Block / scalar transform callable

IEEE division at selected precision

SOFTMAX: CONTRACTS COMPOSE

1 max --> 2 exp --> 3 retain --> 4 sum --> 5 normalize

1 MAXIMUM

$$m = \max_i z_i$$

Named expression:
canonical_block_dyadic<64>

2 SHIFTED EXP

$$e_i = \exp(z_i - m)$$

Block / scalar transform callable

3 RETAIN e_i

$$\mathbf{e} = (e_1, \dots, e_n)$$

Controlled materialization

4 NORMALIZATION SUM

$$s = \sum_{i=1}^n e_i$$

Named expression:
canonical_block_dyadic<64>

5 NORMALIZE

$$y_i = e_i / s$$

Block / scalar transform callable

Reproducible Transcendental Math Functions

What did we actually specify?

A real function name is not a floating-point computation.

exp, sin, cos over the reals -> ? -> floating-point bits

Range reduction, polynomial, evaluation order and intermediate precision are all left open.

The missing part is the numerical expression.

Two ways to get reproducible bits

Route A

Specify the answer.

Every result is the correctly rounded real result.

Route B

Specify the computation.

Every backend evaluates the same named expression.

Both give reproducible bits - Both Routes vectorise

Only Route A also forces the strongest possible accuracy contract.

Accuracy is a separate contract

Two independent questions that are usually answered as one.

How accurate? correctly rounded · faithful · wide, 8 ULP

Must it reproduce? a separate question

I can make any of these reproducible.

If I can have the strongest contract and it is fast enough, I will take it.

So what does loosening accuracy actually buy?

Accuracy is a dial. Cost is a staircase.

Accuracy can vary continuously. The implementation tends to change in steps.

polynomial degree working width lookup regime container

remove a coefficient · change the range reduction · stop carrying another word · resize a table · change the representation

Loosening accuracy only helps when it lets us change the structure.

We asked for 64 ULP instead of 8. The answer got worse and it was not any faster.

Keep going down and the degree reaches zero. We come back to that.

Spend precision where it matters

exp

Change the global structure.

triple-word -> double-word + segmentation

Same 8 ULP contract. Less arithmetic per element.

sin, cos

Change where the work happens.

ordinary arguments -> cheap path

hard reduction -> extra precision

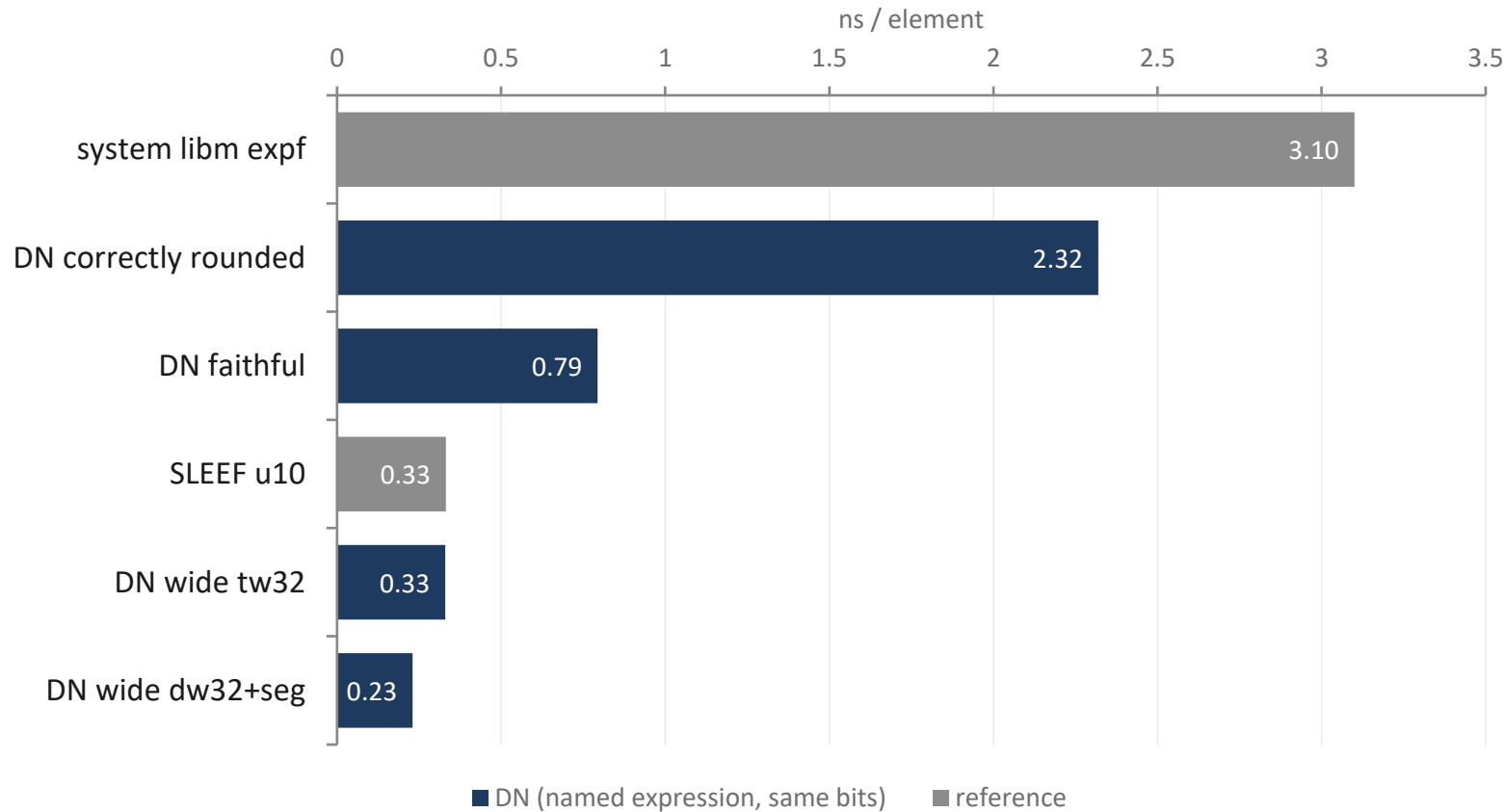
awkward arguments -> fallback

One crosses a step. The other avoids paying for it on every input.

The public contract does not move. Only where the precision is spent.

Put our numbers in context

Measured FP32 exp, AVX-512, ns/element, lower is better. libm row is scalar. DN rows time the hot expression.



CORE-MATH

correct-rounding reference; not a same-chart
SIMD timing row

RLIBM

generation-method reference; rounding
intervals + LP constraints

**The result is not “wide is faster”. It is:
faster useful contract, named expression,
same bits.**

sin / cos, matched AVX-512 run

8 ULP beats SLEEF u10.

SLEEF u35 is still faster.

FP4: the function is the table

all 16 inputs -> correctly rounded results -> 16-entry table -> SIMD permute

There is no approximation problem any more.

Correct rounding becomes easy to verify: you can test the entire domain.

The question is no longer how fast the polynomial evaluates. It is how fast the lookup runs.

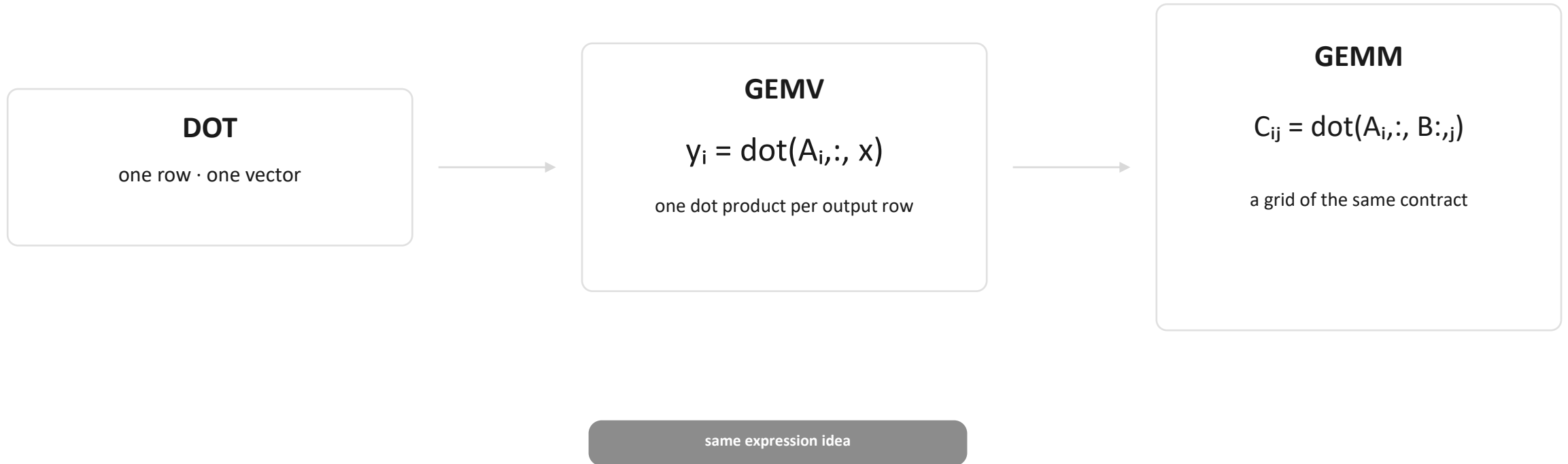
The numerical meaning is fixed. The implementation is not.

Reproducible Linear Algebra

Linear algebra starts with one dot product

GEMV and GEMM are not mysterious once the dot-product expression is named.

$$\text{dot}(a,b) = \text{transform_reduce}(a,b, \times, +)$$



Once the dot product has a named geometry, matrix algebra becomes composition.

The LLM witness stayed close to llama2.c

First measure the cost of determinism without quietly replacing the application.

Karpathy-style matmul

```
// W(d,n) @ x(n) -> xout(d)
for (i = 0; i < d; ++i) {
    float val = 0.0f;
    for (j = 0; j < n; ++j)
        val += w[i*n+j] * x[j];
    xout[i] = val;
}
```

same model · same weight layout · same row decomposition



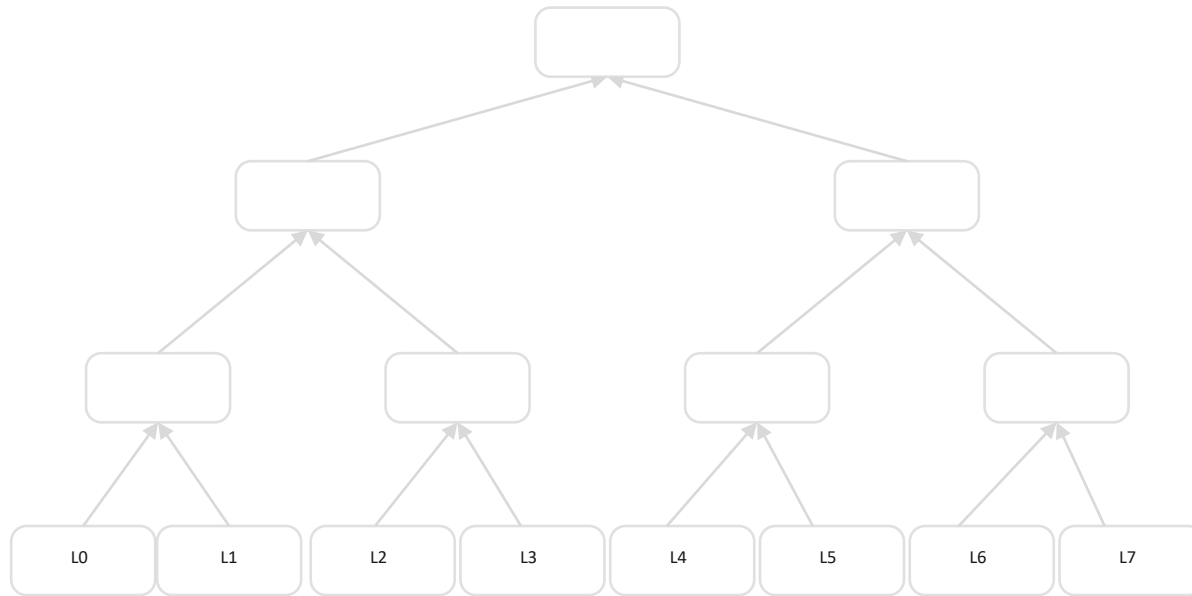
change what the dot product means

not the shape of the neural network

Two measurements: cost of determinism first; performance ceiling second.

First serious GEMM expression: block-dyadic

A numerically attractive contraction: fixed leaves, then a prescribed dyadic tree over K.



K leaves: each leaf is a local ascending FMA chain

Why we liked it

- ISA-width independence
- recursive/cache-oblivious shape
- natural threading
- pairwise-style error behaviour
- complete output matrix is the observation

The tree gives a beautiful numerical contract.

On CPU, the tree was good enough

The deterministic expression did not produce a persistent large-matrix CPU penalty.



On CPU, the hardware was happy with the tree.

Interpretation: small performance differences, not a show-stopper.

Then CUDA pushed back

The same numerical idea remained correct, but the physical cost changed.

RTX 4090 / Ada FP32 GEMM



~67% of cuBLAS

What changed?

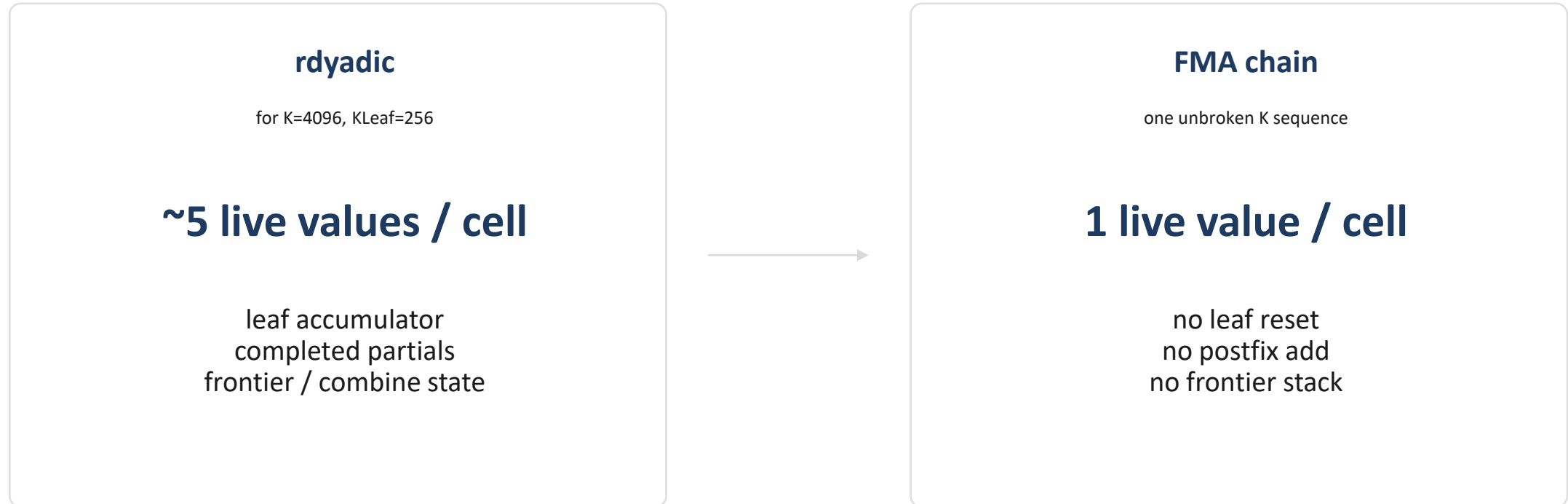
The bits were right.
The contract travelled.
But the tree had physical weight:

**registers · local frames · frontier bookkeeping
· occupancy**

The numerical geometry was portable. The cost of representing it was not.

Suspect: expression state has physical weight

The tree carries partials; the chain carries only the live output accumulator.



Not proof of every percentage point — but exactly the mechanism the GPU literature warns about.

Turn the dial: ascending-K FMA chain

More sequential as mathematics; cheaper as machine state.

Numerical contract

$$a_0 = +0$$

$$a_{k+1} = \text{fma}(A_{ik}, B_{kj}, a_k)$$

$k = 0 \dots K-1$, strictly ascending

Blocking , tile the execution , not the expression

```
for slab in K_slabs:  
    load A/B slab  
    for kk in slab:  
        acc = fma(..., acc)
```

The accumulator crosses the slab boundary.

You don't have to evaluate the chain in a single pass, you can stop and start, provided that you resume with the appropriate state.

Result: the chain is faster, but the tree is nicer

This is not “one true GEMM”. It is an explicit expression-geometry trade-off.

Block-dyadic

error bound: $O(u \log K)$

more expression state
excellent CPU story

FMA chain

worst case: $O(u K)$

much lighter state
~90% cuBLAS on Ada

Parallelism in the expression and parallelism in the machine are not the same thing.

The expression is fixed. The realization is free.

Conclusion

Expression geometry

The shape of the expression determines more than the result.

Reproducibility	which operations and dependencies are fixed
Accuracy	depth and rounding-error propagation
Work and span	dependency structure
Streamability	whether the computation continues incrementally
Live state	intermediate values that must be retained
SIMD and ILP	independent operations available
Threading	independent subexpressions available
GPU efficiency	state, occupancy, reduction structure

One geometry, chosen once, applied at every level:

reduce · scan · softmax · dot · GEMV · GEMM · llama2.c

The same expression contract at each level is what lets determinism pass through composition. Which geometry is worth choosing depends on the machine: the same tree that costs nothing on a CPU is register pressure on a GPU.

What a deterministic numerical contract specifies

Elementary operation semantics

What each operation on the finite objects means. IEEE 754 provides much of the basis for basic arithmetic; where it is insufficient, the numerical contract must say more.

Expression or exact state

Either the floating-point dependency structure to be evaluated, or an exact intermediate state from which the result is determined.

Required observations

Which results of the computation are observable: reduction roots, scan prefixes, matrix outputs, application boundaries.

$$\text{reduce}(x) = \text{inclusive_scan}(x)[n-1]$$

Current contracts do not require this relationship to hold bit for bit.

These properties survive composition when a composer preserves the invariants on which the required observations depend. That is what allows deterministic primitives to form deterministic algorithms.

Specified, and free

Specified

representation and operation semantics

expression or exact state

required observations

consistency between observations

Free

SIMD width

thread count

scheduling

partitioning

tiling and blocking

register and memory strategy

CPU or GPU realization

Same contract, four toolchains, three vendors, different realizations, identical bits.

C++29: use the hardware, check the bits

We can specify deterministic parallel numerical computation.

It composes.

It can be fast.

Specify numerical algorithms in terms of the finite-precision expressions or exact states they compute, and the observations required from them.

Leave the physical realization free.

P4016R0 canonical reduction · P4229R0 canonical scan

Contact Details

andrew@determinuslabs.com

Papers and examples scan and reduce:

<https://github.com/andyD123/canonical-reduce>